02 - Fundamentos de Linux

Introducción

Fundamentos de Linux - Cómo usar Linux

Linux es un sistema operativo de código abierto que se destaca por su potencia y versatilidad, siendo una herramienta útil tanto para usuarios principiantes como para aquellos con más experiencia en informática. En este capítulo vamos a ver los fundamentos del uso de Linux, centrándonos en la interfaz de línea de comandos (CLI), que a menudo se conoce como la terminal, imprescindible para hacking Wi-Fi. En el curso vamos a utilizar Linux para los ejercicios en la VM (Máquina Virtual - Virtual Machine) del laboratorio, específicamente una distribución Ubuntu.

Comenzando con la Terminal

La terminal es una interfaz de texto para interactuar con el ordenador. A diferencia de la interfaz gráfica de usuario (GUI) que la mayoría de los usuarios están acostumbrados, la terminal proporciona una forma más directa y potente de controlar tu ordenador.

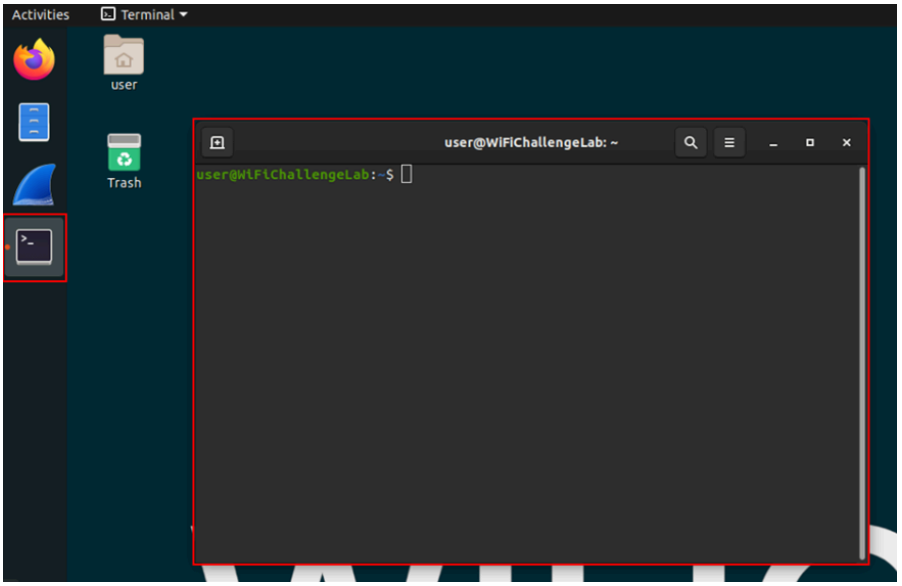
Todos los ejercicios se realizan en WiFiChallenge Lab.

Abrir la Terminal

- **En Ubuntu**: Usa el atajo Ctrl + Alt + T o busca Terminal en el menú de aplicaciones. En el Laboratorio WiFiChallenge hay un atajo en la barra lateral izquierda. También se puede pulsar en la esquina superior izquierda Activities y escribir Terminal.

Ejercicio: Arranca WiFiChallenge Lab y abre la terminal.

Solución:



Abrir terminal en WiFiChallenge Lab

Comprender el Prompt de Comandos

Al abrir la terminal, verás un prompt, que típicamente termina en un signo de dólar ($) para usuarios generales o un signo de almohadilla (#) para el usuario root. Este prompt incluye información como tu nombre de usuario, nombre del sistema, y el directorio de trabajo actual. En este caso, vamos a trabajar con 2 usuarios: user y root. Por defecto la terminal se inicia en el directorio del usuario actual, representado con la virgulilla (~). Esta es la carpeta principal asociada al usuario, en el caso de user es /home/user y en el caso de root es /root.

Comandos Básicos

Navegar por el Sistema de Archivos

- pwd (print working directory): Muestra el directorio actual.
- ls (list): Muestra archivos y directorios en el directorio actual. Podemos utilizar ls -a para ver archivos ocultos y ls -l para información detallada.
- cd (change directory): Cambia el directorio actual. cd .. sube un nivel de directorio, mientras que cd lleva a la carpeta de inicio del usuario, su $HOME. Para volver a la carpeta anterior, cd ..

Ejercicio: Ve a la carpeta '/tmp' y lista los archivos allí.

Solución:

```
user@WiFiChallengeLab: /tmp
user@WiFiChallengeLab:~$ cd /tmp/
user@WiFiChallengeLab:/tmp$ ls
config-err-StuEeA
nzyme-alerts-user.log
ssh-GfmPIOb9mmlt
systemd-private-41587b5acc3f4e969b254fdb0e74fb22-colord.service-wH803e
systemd-private-41587b5acc3f4e969b254fdb0e74fb22-haveged.service-9JROZe
systemd-private-41587b5acc3f4e969b254fdb0e74fb22-ModemManager.service-lYDHdj
systemd-private-41587b5acc3f4e969b254fdb0e74fb22-switcheroo-control.service-Emty
Pg
systemd-private-41587b5acc3f4e969b254fdb0e74fb22-systemd-logind.service-8C46qj
systemd-private-41587b5acc3f4e969b254fdb0e74fb22-upower.service-cPQhhg
tracker-extract-files.1001
tracker-extract-files.130
update_wifi_lab.log
user@WiFiChallengeLab:/tmp$
```

Ejercicio ls y cd

Gestionar Archivos y Directorios

- mkdir (make directory): Crea un nuevo directorio.
- rmdir (remove directory): Elimina directorios. Solo es posible eliminar un directorio si está vacío.
- rm (remove): Elimina archivos o directorios. Úsalo con precaución; rm -r elimina directorios y su contenido. Ten mucho cuidado al usarlo con el usuario root (administrador).
- mv (move): Mueve o renombra archivos y directorios.
- cp (copy): Copia archivos o directorios. Usa cp -r para copiar directorios y archivos dentro.

Ejercicio: Crea una carpeta "test" en /tmp, copia el archivo /home/user/restartWiFi.sh a la carpeta y luego elimina toda la carpeta.

Solución:

```
user@WiFiChallengeLab: /tmp
user@WiFiChallengeLab:/tmp$ mkdir test
user@WiFiChallengeLab:/tmp$ cp /home/user/restartWiFi.sh test/
user@WiFiChallengeLab:/tmp$ ls test/
restartWiFi.sh
user@WiFiChallengeLab:/tmp$ cd test/
user@WiFiChallengeLab:/tmp/test$ ls
restartWiFi.sh
user@WiFiChallengeLab:/tmp/test$ cd ..
user@WiFiChallengeLab:/tmp$ rm -r test/
user@WiFiChallengeLab:/tmp$
```

Ejercicio mkdir, cd y rm

Comprender su

El comando su (substitute user) en sistemas tipo UNIX se usa para cambiar el contexto del usuario actual a otro usuario durante una sesión de terminal. A diferencia de sudo, su cambia al usuario root o a un usuario especificado y requiere la contraseña del usuario objetivo.

- Para cambiar al usuario root, simplemente ejecuta su - root y proporciona su contraseña.
- Es posible cambiar a otro usuario especificando su nombre de usuario, como su - user, proporcionando su contraseña.
- Por defecto, su cambia al usuario root y requiere el entorno respectivo del usuario. Para heredar el entorno del usuario original, usa su -.

su puede ser especialmente útil para tareas administrativas cuando se necesita acceso prolongado al usuario root, o si un script está escrito desde la perspectiva de otro usuario. Sin embargo, a menudo se recomienda usar sudo para tareas administrativas que requieren permisos elevados solo momentáneamente debido a los riesgos de seguridad asociados con el acceso completo al root.

Usando sudo

sudo (superuser do) te permite ejecutar comandos en el contexto de otro usuario, por defecto, el usuario root. Es esencial para tareas administrativas como instalar software, configurar o modificar archivos del sistema operativo.

- Para usar sudo, antecede tu comando con sudo e ingresa tu contraseña cuando se te pida. Por ejemplo, sudo apt-get update actualiza tu lista de paquetes en sistemas basados en Debian.
- Se puede ejecutar sudo su para ejecutar su cómo root y abrir una sesión como root sin la contraseña de root

sudoers

El comportamiento y la configuración de sudo están determinados por el archivo /etc/sudoers. Este archivo controla qué usuarios pueden usar sudo y cuáles son los permisos específicos que tienen.

El archivo /etc/sudoers se estructura en reglas que especifican el acceso a sudo. Aquí hay algunas configuraciones comunes:

- **Incluir usuarios en sudoers**: Puedes agregar un usuario al archivo sudoers usando visudo (una herramienta segura para editar este archivo que previene errores de sintaxis). Por ejemplo:

user ALL=(ALL:ALL) ALL

Esta línea le da a user permisos para ejecutar cualquier comando como cualquier usuario en cualquier máquina (en configuraciones distribuidas).

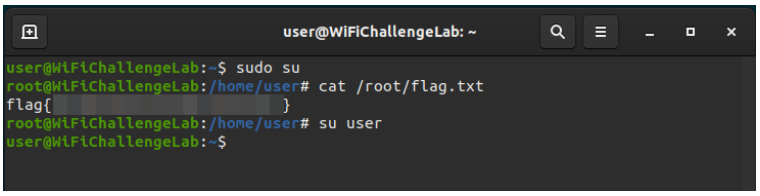
- **Permisos específicos**: También puedes dar permisos específicos a un usuario o grupo. Por ejemplo:

user ALL=/usr/bin/apt-get

En el caso del laboratorio, el usuario root no tiene contraseña habilitada, por lo que no es posible acceder a él usando su, pero si podemos utilizar sudo su para ejecutar su por defecto como root, accediendo a una Shell como administrador, ya que el usuario user es miembro de sudoers.

Ejercicio: Cambia a usuario root y lee la primera flag del challenge 0 de WiFiChallenge Lab, luego cambia a user usando su

Solución:



Ejercicio sudo su

Comandos Avanzados

tmux

tmux es una herramienta poderosa que te permite crear, gestionar y navegar por múltiples sesiones de terminal desde una sola pantalla. Es especialmente útil para trabajo remoto, tareas de larga duración y flujos de trabajo de desarrollo complejos.

Instalación y uso básico

Normalmente se puede instalar usando el gestor de paquetes de tu sistema:

- En Ubuntu/Debian: sudo apt-get install tmux
- En macOS: brew install tmux (usando Homebrew)

Para iniciar una nueva sesión de tmux:

tmux new -s [nombre-de-sesión]

Reemplaza [nombre-de-sesión] con el nombre deseado para la sesión. Si no especificas un nombre, tmux le asignará un nombre numérico.

Navegando en tmux

Una vez dentro de una sesión de tmux, se puede ver una barra de estado en la parte inferior, mostrando el nombre de la sesión, ventanas, y más información.

Los comandos de tmux se invocan con una tecla de prefijo seguida de una tecla de comando. El prefijo predeterminado es Ctrl-b, aunque muchos usuarios lo reasignan a Ctrl-a por conveniencia.

- Para crear una nueva ventana: prefijo + c
- Para cambiar entre ventanas: prefijo + p (anterior) o prefijo + n (siguiente)
- Para renombrar la ventana actual: prefijo + ,
- Para ver el árbol de ventanas: prefijo + w
- Para cerrar una ventana: prefijo + &. Pedirá confirmación antes de proceder.

Dividiendo Ventanas en Paneles

- Una de las características más poderosas de tmux es la capacidad de dividir ventanas en múltiples paneles. Para dividir la ventana verticalmente: prefijo + %
- Para dividir la ventana horizontalmente: prefijo + "
- Para navegar entre paneles: prefijo + tecla de flecha
- Para cerrar el panel actual: prefijo + x
- También puedes salir de la Shell con Ctrl+d, o tecleando exit
- Para hacer zoom en un panel: prefijo + z, misma secuencia para volver.

Desconectando y Volviendo a Conectar Sesiones

Una de las grandes ventajas de tmux es que mantiene las sesiones en ejecución en segundo plano incluso si te desconectas de ellas.

- Para desconectar de una sesión: prefijo + d
- Para listar sesiones en ejecución: tmux ls
- Para reconectar a una sesión: tmux attach -t [nombre-de-sesión]

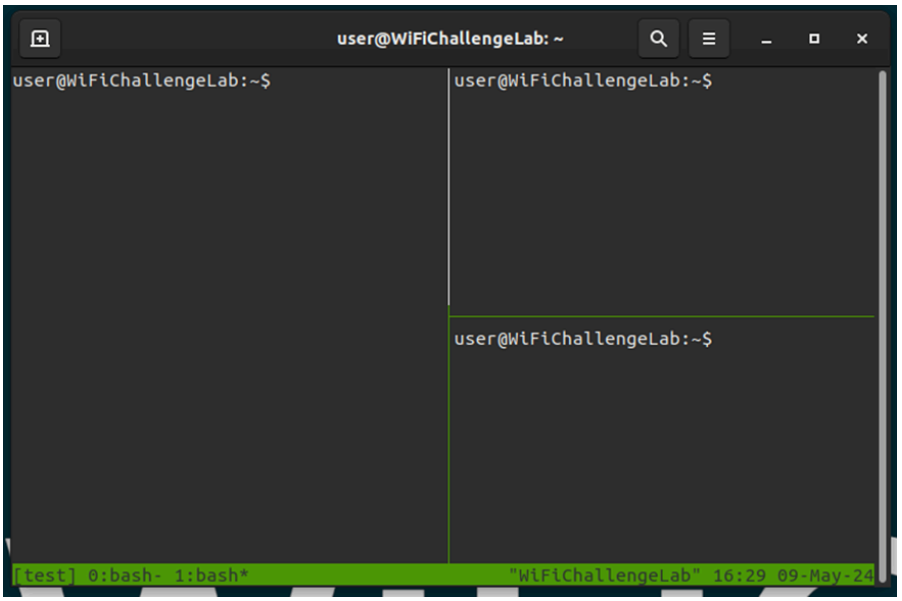
Gestión de Sesiones

Se pueden gestionar sesiones de manera más efectiva usando algunos comandos:

- Para eliminar una sesión: tmux kill-session -t [nombre-de-sesión]
- Para renombrar una sesión: prefijo + $

Ejercicio: Abre una sesión de tmux llamada test, crea una nueva ventana y divide la pantalla verticalmente, luego divide la derecha horizontalmente.

Solución:



Ejercicio tmux

Comprendiendo los Permisos de Archivos en Linux

En Linux, cada archivo tiene permisos de acceso específicos asignados para tres categorías distintas de usuarios:

- **Propietario**: El usuario que creó el archivo.
- **Grupo**: Usuarios que están agrupados para propósitos administrativos. Cada archivo pertenece a un único grupo.
- **Otros**: Cualquier otro usuario que tiene acceso al archivo.

Tipos de Permisos

Hay tres tipos de permisos que Linux permite para cada archivo o directorio:

- **Lectura (r)**: Permiso para abrir y leer el archivo. Para directorios, este permiso permite listar el contenido del directorio.
- **Escritura (w)**: Permiso para modificar el archivo. Para directorios, este permiso permite agregar, eliminar y renombrar archivos almacenados en el directorio.
- **Ejecución (x)**: Permiso para ejecutar el programa o script. Para directorios, este permiso permite acceder al contenido y metainformación sobre el archivo o directorio.

Visualizando Permisos

Para ver los permisos de archivos y directorios, usa el comando ls -l. Aquí hay un desglose de lo que podría parecer el resultado:

-rwxr-xr-x 5 nombre_usuario nombre_grupo tamaño fecha hora nombre_archivo

- El primer carácter representa el tipo de archivo (- para archivos normales, d para directorios).
- Los siguientes tres caracteres (rwx) muestran los permisos para el propietario.
- Los siguientes tres (r-x) para el grupo.
- Los últimos tres (r-x) para otros.

Modificando Permisos

Los permisos pueden ser cambiados usando el comando chmod (change mode), que puede ser utilizado en modo simbólico o numérico:

- **Modo simbólico**: Usa letras (r, w, x, -) y símbolos (+, -) para modificar los permisos.
  - Ejemplo: Para añadir permiso de ejecución para todos los usuarios, usa chmod a+x nombre_archivo.
  - Ejemplo: Para quitar el permiso de escritura para el grupo, usa chmod g-w nombre_archivo.
- **Modo numérico**: Usa valores numéricos que representan permisos en . Cada uno de los tres dígitos corresponde a una categoría de usuarios (propietario, grupo, otros), haciendo referencia a las columnas en binario:
  - 4: lectura (r)
  - 2: escritura (w)
  - 1: ejecución (x)
- Los permisos son una suma de estos valores:
  - Ejemplo: chmod 755 nombre_archivo
    - 7 (propietario) = 4+2+1 (lectura + escritura + ejecución)
    - 5 (grupo) = 4+0+1 (lectura + ejecución)
    - 5 (otros) = 4+0+1 (lectura + ejecución)

Hay un sitio web para calcular fácilmente estos valores:

Permisos Especiales

Hay algunos permisos especiales que también pueden ser configurados en archivos Linux:

- Setuid **(s)**: Cuando se establece en un archivo ejecutable, permite que el archivo sea ejecutado con los permisos del dueño del archivo.
- Setgid **(s)**: En directorios, los archivos creados dentro del directorio heredan su grupo del directorio, no del usuario que lo crea.
- Sticky Bit **(t)**: Principalmente usado en directorios. Restringe la eliminación de archivos, permitiendo solo al dueño del archivo o al root eliminar archivos dentro del directorio.

Comprender y gestionar los permisos de archivos en Linux es crucial para asegurar tu sistema y gestionar quién tiene acceso a qué archivos. Siempre utiliza la precaución al establecer permisos, especialmente con el permiso de ejecución, para evitar riesgos de seguridad no intencionados.

Ejercicio: Establece para el archivo flag.txt los siguientes permisos: Propietario: Leer, Escribir y ejecutar; Grupo: ejecutar; Público: leer (rwx--xr--)

Solución:

```
root@WiFiChallengeLab:~# chmod 714 flag.txt
root@WiFiChallengeLab:~# ls -la flag.txt
-rwx--xr-- 1 root root 32 Mar 14  2023 flag.txt
root@WiFiChallengeLab:~#
```

Ejercicio chmod

Buscar y Filtrar Archivos

find Busca archivos en una jerarquía de directorios. Ejemplo: find /home -name "*.txt" encuentra todos los archivos de texto en el directorio /home.

grep: Busca patrones dentro de archivos. Ejemplo: grep 'ejemplo' nombre_archivo busca la palabra "ejemplo" dentro del nombre_archivo.

Ejercicio: Encuentra los 2 archivos body.html en el directorio herramientas (/root/tools/).

Solución:

```
user@WiFiChallengeLab: ~
root@WiFiChallengeLab:~# find /root/tools/ -name body.html
/root/tools/eaphammer/core/wskeyloggerd/templates/user_defined/login/body.html
/root/tools/eaphammer/core/wskeyloggerd/templates/user_defined/download/body.html
root@WiFiChallengeLab:~#
[test] 0:bash- 1:bash*                    "WiFiChallengeLab" 16:36 09-May-24
```

Ejercicio find

SCP

SCP te permite transferir archivos de forma segura entre sistemas a través de SSH, encriptando tanto los datos como las credenciales.

Sintaxis de SCP

scp [opciones] archivo_origen usuario@host_remoto:ruta_de_destino

Ejemplos

- **Copiar un archivo a un servidor remoto**

scp miarchivo.txt usuario@192.168.1.10:/home/usuario/

- **Copiar un archivo desde un servidor remoto**

scp usuario@192.168.1.10:/home/usuario/miarchivo.txt /ruta/local/

- **Copiar recursivamente un directorio**

scp -r /directorio/local usuario@remoto:/ruta/hacia/destino/

Opciones comunes de SCP

- -r: Copia recursivamente directorios completos
- -P: Especifica un puerto personalizado
- -C: Habilita la compresión para transferencias más rápidas

Túneles SSH

Descripción

Los túneles SSH (del inglés, Secure Shell tunnels), se utilizan para redirigir, y asegurar el tráfico entre dos puntos a través de un túnel cifrado. Este método es común para el reenvío de información a través de redes no cifradas, garantizando la privacidad y seguridad.

¿Por qué usar Túneles SSH?

Los túneles SSH son útiles para:

- Reenviar tráfico no seguro a través de canales cifrados.
- Saltarse políticas de firewall que bloquean ciertos puertos.
- Acceder de manera segura a una red remota.

En el examen no es obligatorio, pero si mucho más cómodo saber realizar túneles para acceder remotamente a webs desde el navegador local.

Tipos de Túneles SSH

- **Reenvío Local**: Reenvío de un puerto en el lado del cliente a una aplicación del lado del servidor.
- **Reenvío Remoto**: Permite el acceso a un servicio local desde un servidor remoto.
- **Reenvío de Puerto Dinámico**: Permite configurar un servidor proxy tipo SOCKS que reenvía el tráfico a través del cliente SSH.

Reenvío Local (Túnel Local)

El reenvío local, también llamado túnel SSH local, reenvía el tráfico desde la máquina cliente a la máquina servidor a través de un canal cifrado. Es útil para acceder a un servicio remoto como si estuviera ejecutándose en la máquina local.

Escenario

Supongamos que se quiere acceder a un servidor de base de datos que escucha en el puerto 3306 en un servidor remoto, pero el puerto no está expuesto a Internet por razones de seguridad. Es posible crear un túnel desde un puerto local al puerto del servidor remoto.

Comando de Configuración

```
ssh -L [puerto_local]:localhost:[puerto_remoto] [usuario]@[servidor_remoto]
#Ejemplo
ssh -L 3306:localhost:3306 usuario@ejemplo.com
```

Este comando asigna el puerto 3306 en tu máquina local al puerto 3306 en el servidor remoto ejemplo.com a través de una conexión SSH. Las aplicaciones en tu máquina local pueden usarlocalhost:3306 para acceder al servidor de base de datos de manera segura.

No es necesario que ambos puertos sean iguales.

Reenvío Remoto (Túnel Remoto)

El reenvío remoto es el inverso del reenvío local. Permite a los usuarios reenviar un puerto desde el lado del servidor a un servicio en el lado del cliente.

Escenario

Supongamos que hay un servidor web ejecutándose en el puerto 8080 de la máquina local y se quiere que un cliente que trabaja en un servidor remoto acceda a este servidor web sin exponerlo a Internet.

Comando de Configuración

```
ssh -R [puerto_remoto]:localhost:[puerto_local] [usuario]@[servidor_remoto]
#Ejemplo
ssh -R 9090:localhost:8080 usuario@ejemplo.com
```

Este comando asigna el puerto 9090 en el servidor remoto (ejemplo.com) al puerto 8080 en tu máquina local. El servidor remoto puede acceder a tu servidor web local visitandolocalhost:9090.

Reenvío Dinámico (Proxy SOCKS)

El reenvío de puerto dinámico convierte al cliente en un servidor proxy SOCKS. Un proxy SOCKS es un servidor que intercambia paquetes de red entre un cliente y un servidor a través de un proxy, permitiendo la transferencia de datos en diferentes protocolos sin necesidad de conocer la aplicación específica. Permite que el cliente reenvíe tráfico dinámicamente desde múltiples puertos, a diferencia del reenvío local y remoto, que funcionan con puertos específicos.

Escenario

Supongamos que se quiere navegar por Internet de forma segura desde una ubicación remota, asegurando que todos los datos de navegación estén cifrados y potencialmente sorteando restricciones geográficas.

Comando de Configuración

ssh -D [puerto_SOCKS_local] [usuario]@[servidor_remoto]

Ejemplo:

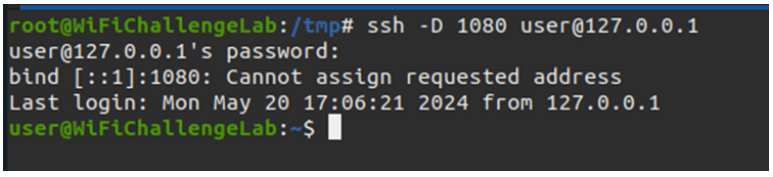
ssh -D 1080 usuario@ejemplo.com

Este comando crea un proxy SOCKS enlocalhost:1080. Es posible configurar tu navegador u otras aplicaciones que admitan proxy para usar este proxy y todo el tráfico se redirigirá dinámicamente a través de la conexión SSH. Generalmente un navegador como Firefox.

Ejercicio: Conéctate desde la VM a sí misma creando un túnel dinámico y configura Firefox para utilizar el SOCKS. (Esto lo hacemos para simular una conexión a un servidor remoto sin necesidad de acceder a otro servidor, en un caso real se sustituiría 127.0.0.1 por la IP del servidor remoto)

Solución:

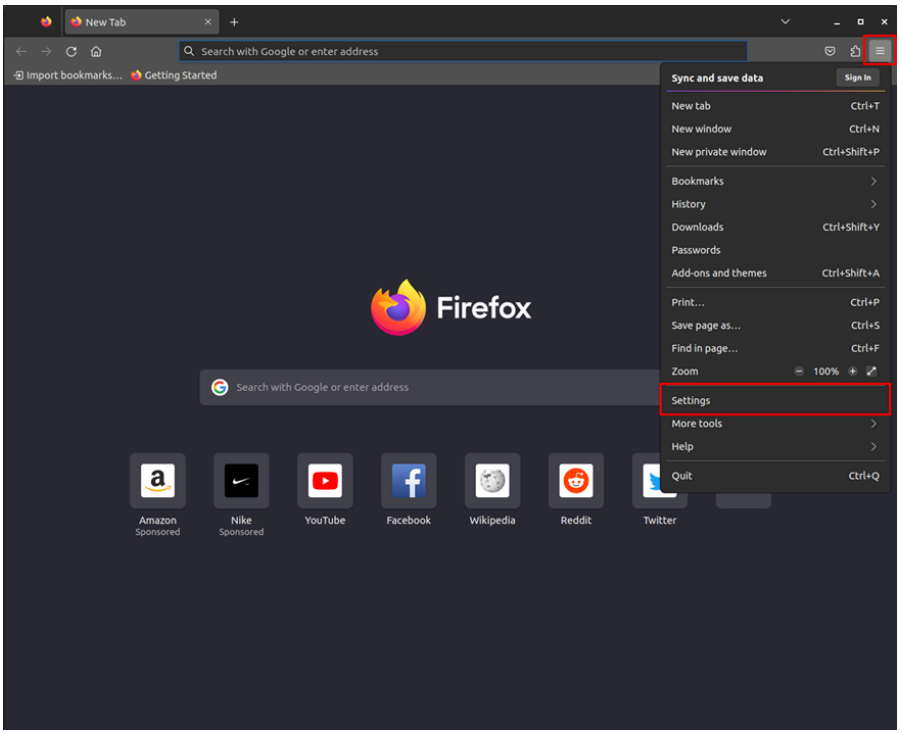
ssh -D 1080 user@127.0.0.1



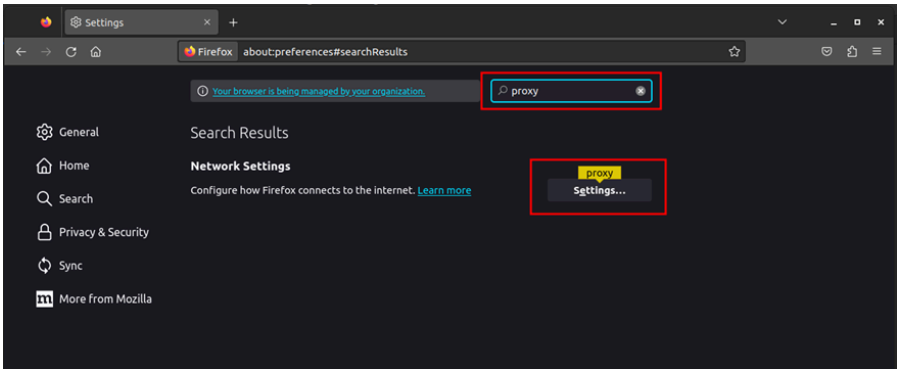
Solución reto túnel

El error: bind [::1]:1080: Cannot assign requested address no es importante, ya que es un error de intentar publicar el puerto en IPv6

Después de haber abierto el túnel dinámico, sigue los pasos A continuación, para configurar Firefox para utilizar el proxy SOCKS:



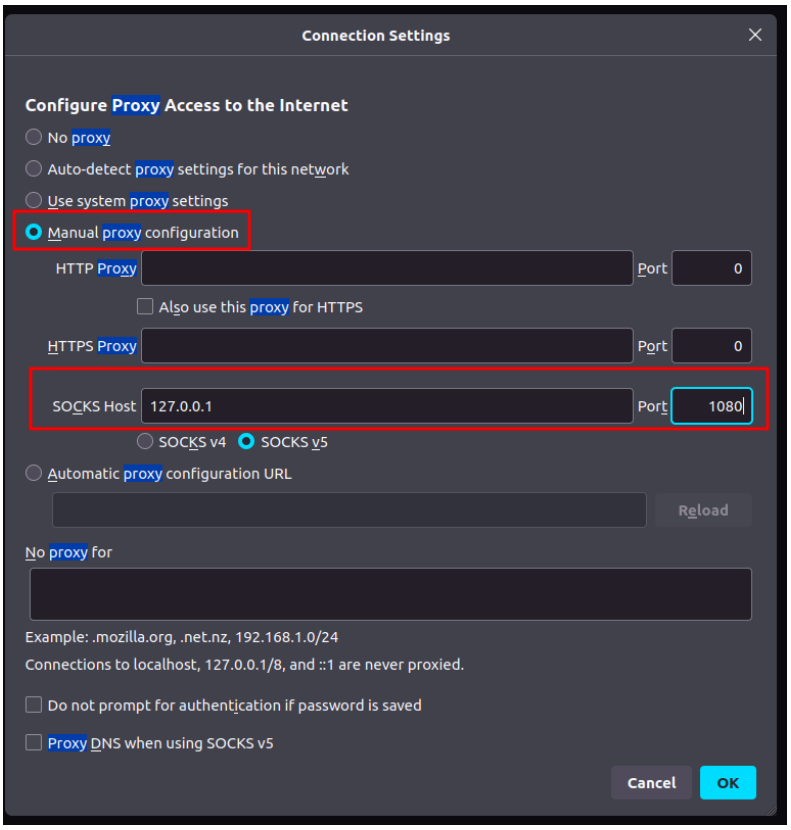
Configuración de Firefox



Configuración de proxy en Firefox

- Haz clic en el botón de menú (tres líneas horizontales en la esquina superior derecha).
- Selecciona Settings o Preferencias
- En la parte superior buscamos proxy
- Y pulsamos en Settings

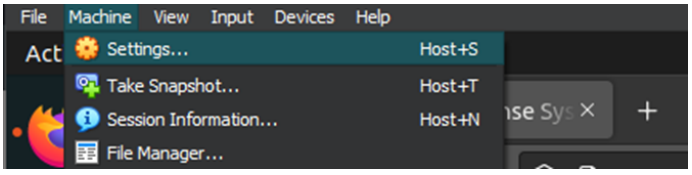
Una vez en la configuración del proxy podemos configurarlo manualmente con SOCKS v5, la IP 127.0.0.1 que es nuestro propio equipo y el puerto que hemos utilizado en el túnel SSH, en este caso 1080. Y pulsamos OK para guardar.



Configuración de proxy a localhost en el puerto 1080

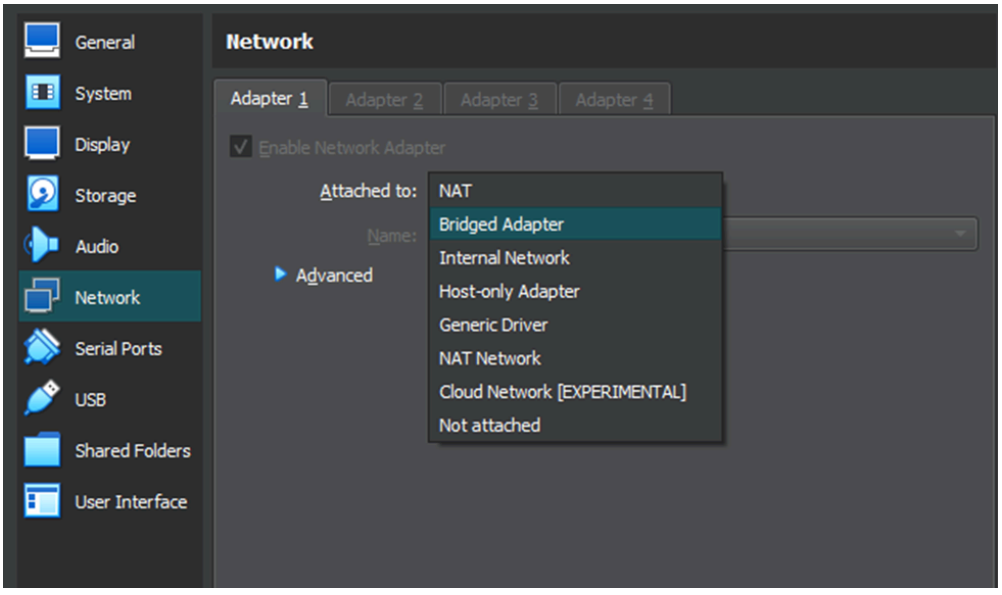
Ejercicio avanzado guiado (opcional): Cambia la VM a modo bridge para estar en la misma red local que el equipo host y haz un túnel remoto para acceder al servicio que tiene la VM en local.

Para hacer esto, primero ponemos en modo Bridge la VM, que en VirtualBox sería simplemente Machine, Settings.



Configuración de la VM

Y ahora Network y cambiar NAT por Bridged Adapter.

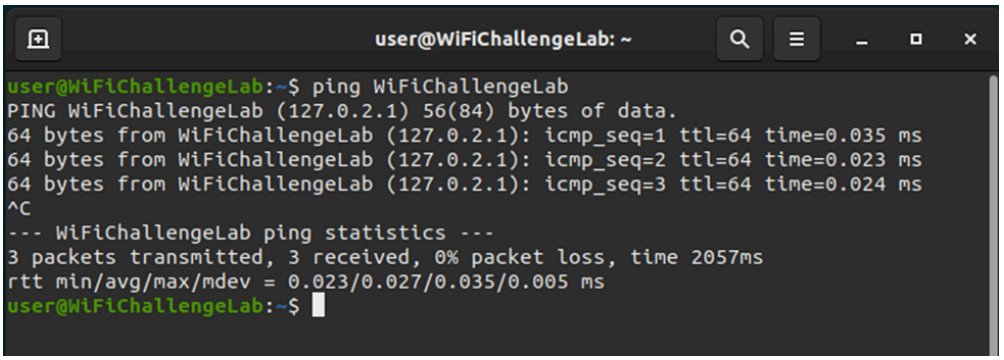


Cambio de configuración de red a Bridge en VBox

Ahora verificamos la IP de la VM y vemos que ha cambiado a una, normalmente del segmento 192.168.1.0.

Ahora, a modo de prueba de concepto podemos levantar un servicio HTTP solo accesible desde la propia VM (utilizamos 127.0.2.1 ya que es la IP que marca el nombre de la máquina si hacemos ping), sin que sea accesible desde la red. Esto lo podemos hacer con Python.

python3 -m http.server 8080 --bind 127.0.2.1



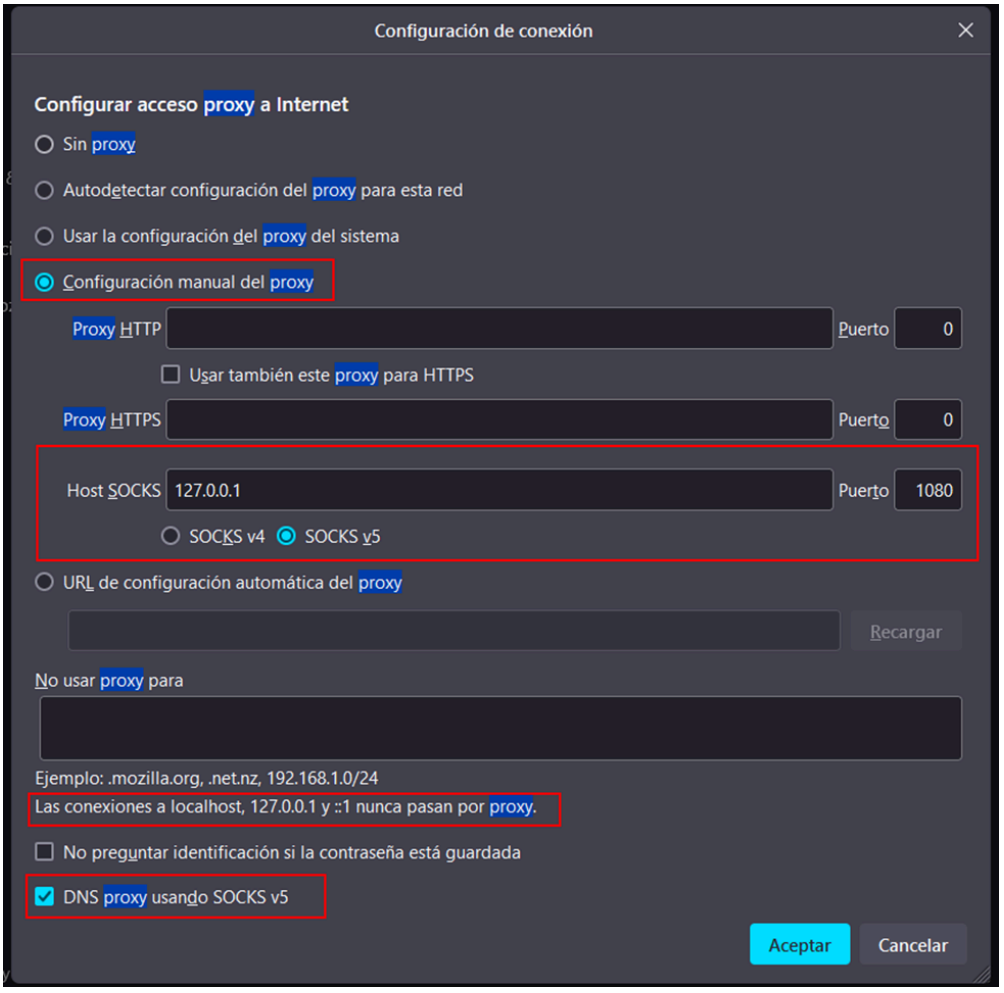
Ping a nombre de maquina

Esto crea un servicio HTTP muy útil para compartir ficheros, ya que comparte la carpeta desde la que se ejecuta y se puede acceder desde cualquier navegador (por lo que es importante no ejecutarlo en una carpeta con secretos o cosas importantes si se publica en la red).

Ahora hacemos el mismo proceso que antes únicamente cambiando el comando de SSH, aquí vamos a usar 192.168.1.100 como ejemplo, pero habría que Reemplazarlo por la IP de la VM.

ssh -D 1080 user@192.168.1.100

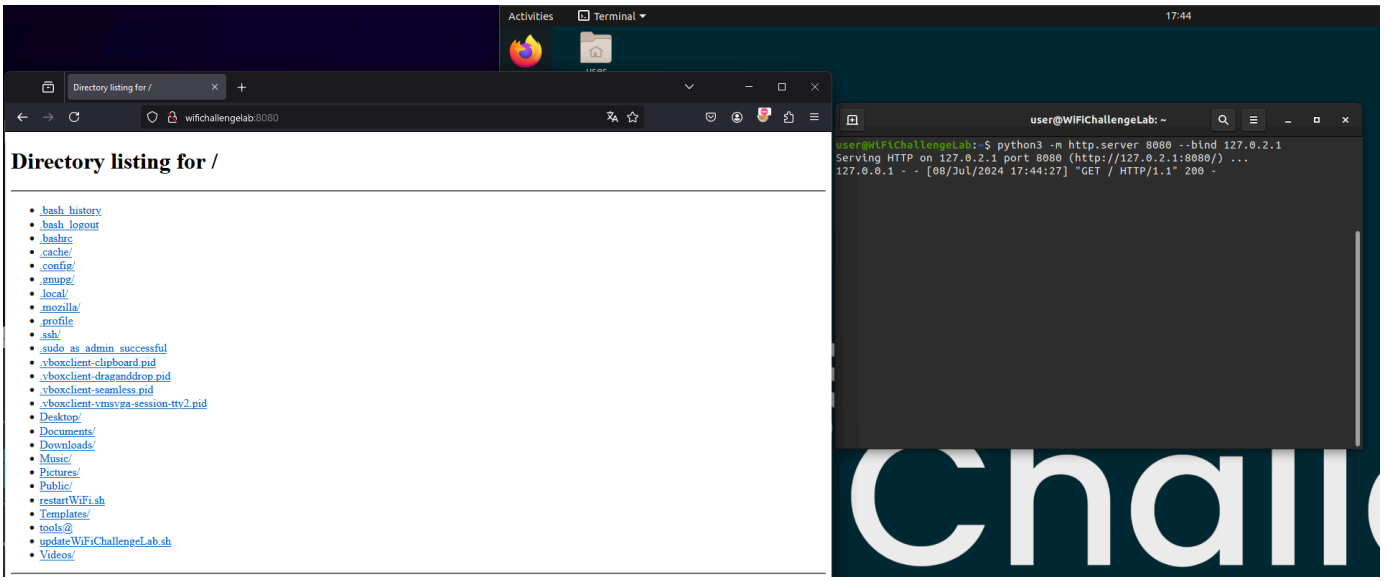
Ahora configuramos Firefox igual, pero añadiendo el Check de DNS por el proxy para que el servidor remote resuelva las peticiones DNS. Esto lo hacemos para esta prueba, porque las conexiones a localhost, 127.0.0.1, etc. Nunca pasan por el proxy y como el servicio solo está en localhost necesitamos usar el nombre del equipo como alias.



Configuración del proxy

Y accedemos a la web de la IP de la VM con el puerto 8080 utilizando el nombre de la máquina como URL.

http://wifichallengelab:8080/



Acceso a servidor a través del proxy

X11 Forwarding

X11 forwarding permite ejecutar aplicaciones gráficas en un servidor remoto mientras la interfaz se muestra en la máquina local. SSH asegura el protocolo X11 encapsulándolo en un túnel cifrado.

X11 Forwarding

- Instala un servidor X en tu sistema local (por ejemplo, XQuartz para macOS, Xming para Windows).
- Asegúrate de que la línea X11Forwarding esté habilitada en el archivo de configuración del servidor SSH (/etc/ssh/sshd_config).
- Usa la opción -X al conectarte:

ssh -X usuario@host_remoto

Ejemplo:

Ejecuta una aplicación gráfica (como firefox) en el servidor remoto:

firefox

La ventana de la aplicación aparecerá en tu máquina local.

Entendiendo el Cracking Wi-Fi: Herramientas y Técnicas

Hashcat

Hashcat es una herramienta de recuperación de contraseñas que admite una amplia gama de algoritmos y es conocida por su velocidad y versatilidad. Utiliza principalmente la potencia de las tarjetas gráficas (GPU) para acelerar el proceso de cracking, aunque también soporta el uso del procesador (CPU).

Cracking MSCHAPv2

MSCHAPv2 es un protocolo de autenticación basado en contraseñas ampliamente utilizado en entornos WPA/WPA2/WPA3-Enterprise. Hashcat se puede utilizar para crackear hashes MSCHAPv2 aprovechando el modo 5500 (Microsoft Challenge Handshake Authentication Protocol v2). El proceso generalmente requiere los datos capturados de la red, donde Hashcat aplica ataques de fuerza bruta o diccionario para adivinar la contraseña.

hashcat -m 5500 hash.txt -a 0 -w 3 rockyou.txt

Este comando indica a Hashcat que crackee el hash en hash.txt utilizando la lista de palabras rockyou.txt, bajo el modo 5500 para MSCHAPv2.

Ejercicio: Crackea el siguiente hash:

user::::ef900eb6993bc7b399b8eaaf63de642e1e1f269f48ed3e22:f48f789635d6aaa4

Cracking WPA/WPA2-PSK con Hashcat

Hashcat también proporciona un mecanismo potente para crackear redes WPA/WPA2-PSK, utilizando el modo 22000. Este es un método eficiente para crackear Handshake WPA/WPA2 capturados. Para usar Hashcat para crackear WPA/WPA2-PSK, primero necesitas un archivo de Handshake, que se puede capturar usando herramientas como aircrack-ng o Wireshark.

hashcat -m 22000 hash.hccapx -a 0 -w 3 rockyou.txt

En este comando:

- -m 22000 especifica el modo para WPA/WPA2-PSK.
- hash.hccapx es el archivo de Handshake en formato hccapx (es posible convertir archivos .cap a .hccapx usando la herramienta cap2hccapx proporcionada por Hashcat).
- -a 0 denota el modo de ataque (directo), utilizando un archivo de diccionario.
- rockyou.txt es el archivo de diccionario utilizado para intentar el cracking.

Este enfoque depende en gran medida de la fuerza del diccionario utilizado. Para mayores tasas de éxito, es aconsejable usar diccionarios completos o utilizar ataques basados en reglas para modificar dinámicamente los diccionarios existentes durante el proceso de cracking.

Aprovechando las capacidades de aceleración de GPU de Hashcat, el cracking de Handshake WPA/WPA2 se vuelve más rápido en comparación con las herramientas basadas en CPU. Esto lo convierte en una opción preferida en escenarios donde la eficiencia de tiempo es crítica.

Ejercicio: Crackea el siguiente hash:

WPA*02*11ede392e66d40d84c6ca5f8982f88fa*02000000010002000000030077696669*cefadaf500e7f55b6ab632bea9f879bbc30d325af2c6886f6560b0380805e0bd*0103007502010a00000000000000000001bf49e050a9794dbce83a4d369b9904dbc1e30c14d2b31e8418790a99161ed4db0000000000000000